

Module 3

Process Coordination and Deadlock

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

Contents

- Process synchronization
- Critical-section problem
- Peterson's solution
- Synchronization hardware
- Semaphores, monitors,
- Classic problems of synchronization.
- System model, deadlock characterization
- Methods for handling deadlocks, deadlock prevention, deadlock avoidance, deadlock detection, recovery from deadlock.

Process Synchronization

- It is the coordination of processes so that they do not interfere with each other when accessing shared resources.
- Why is it needed?
 - Multiple processes may run concurrently in multiprogramming systems.
 - If two processes access a shared variable or file simultaneously, it may lead to data inconsistency.
 - Synchronization ensures mutual exclusion, orderliness, and correctness.

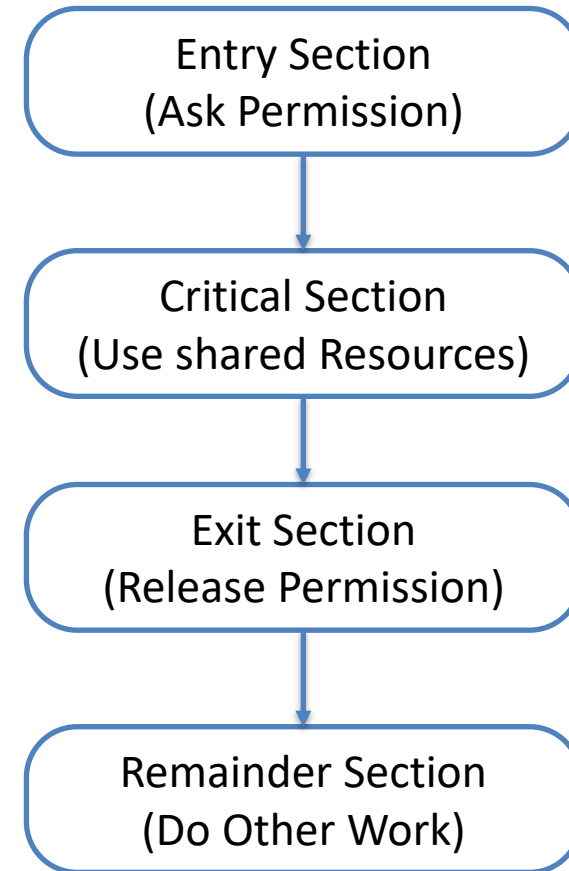
Critical-Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process is in critical section, no other may be in its critical section.
- *Critical section problem* is to design protocol to solve this issue.
- Each process must ask permission to enter critical section in **entry section**, may follow **critical section** with **exit section**, then **remainder section**.

Critical Section

- General structure of process P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```



Critical Section

- A process's program can be divided into **four** parts when we think about the critical section problem:
- **Entry Section**: Before entering the critical section, the process must ask for permission. Make sure only one process enters the critical section at a time.
- **Critical Section**: The actual part of the code where the process uses shared resources (e.g., updates a variable, writes to a file). Only one process can be here at a time.
- **Exit Section**: The process must signal that it's leaving. This allows other waiting processes to enter.
- **Remainder Section**: All other code in the program that does not involve shared resources.

Solution to Critical-Section Problem

- *Mutual Exclusion* - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- *Progress* - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
- *Bounded Waiting* - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.
 - Assume that each process eventually executes.
 - No assumption concerning relative speed of the n processes.

Critical-Section Handling in OS

- Two approaches depending on if kernel is preemptive or non-preemptive.
 - *Preemptive* – allows preemption of process when running in kernel mode.
 - *Non-preemptive* – runs until exits kernel mode, blocks, or voluntarily yields CPU.
 - Essentially free of race conditions in kernel mode.

Peterson's Solution

- It is a software algorithm that solves the Critical Section Problem for two processes.
- It is a two process solution.
- Assume that the *load* and *store* machine-language instructions are atomic; that is, cannot be interrupted.
- **Load** → Copies data from memory into a CPU register.
- **Store** → Copies data from a CPU register into memory
- All computations happen in **CPU registers**, not directly in memory.

Peterson's Solution

- The two processes share two variables:
 - `int turn;`
 - `Boolean flag[2]`
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. *flag[i] = true* implies that process P_i is ready!

Algorithm of Peterson's Solution

- Provable that the three CS requirement are met:

- Mutual exclusion is preserved

- P_i enters CS only if:
either *flag[j] = false* or *turn = i*

- Progress requirement is satisfied.

- Bounded-waiting requirement is met. (Taking turns)

do {

```
flag[i] = true;  
turn = j;  
while (flag[j] && turn == j);
```

critical section

```
flag[i] = false;
```

remainder section

} while (true);

Synchronization hardware

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**.
 - Protecting critical regions via locks.
- Uniprocessors – could disable interrupts.
 - Currently running code would execute without preemption.
 - Generally, too inefficient on multiprocessor systems.
 - Operating systems using this are not broadly scalable.
- Modern machines provide special atomic hardware instructions.
 - Either test memory word and set value.
 - Or swap contents of two memory words.

Solution to CS Problem Using Locks

do {

acquire lock

critical section

release lock

remainder section

} while (TRUE);

test_and_set Instruction

- *test_and_set* is a hardware-supported atomic instruction used to implement mutual exclusion in concurrent systems.

- Definition:

```
boolean test_and_set (boolean *target)  
{  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

- Executed atomically.
- Returns the original value of passed parameter.
- Set the new value of passed parameter to “TRUE”.

Solution using test_and_set()

- Shared Boolean variable lock, initialized to FALSE.
- Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```

compare_and_swap Instruction

- Also known as **CAS**, this atomic instruction is used to **safely update shared memory** in multi-threaded environments **without locks**.

- Definition:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

- Executed atomically.
- Returns the original value of passed parameter “value”.
- Set the variable “value” the value of the passed parameter “new_value” but only if “value” == “expected”. That is, the swap takes place only under this condition.

Solution using compare_and_swap

- Shared integer “lock” initialized to 0; (0 = free, 1 = busy)
- Solution:

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
    /* critical section */  
    lock = 0;  
    /* remainder section */  
} while (true);
```

Mutex Locks

- OS designers build software tools to solve critical section problem.
- Simplest is mutex lock.
- Protect a critical section by first **acquire()** a lock then **release()** the lock
 - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic.
 - Usually implemented via hardware atomic instructions.
- But this solution requires **busy waiting**.
 - This lock therefore called a **spinlock**.

acquire() and release()

- **acquire()** {
 while (!available)
 ; /* busy wait */
 available = false;;
}

- **release()** {
 available = true;
}

do {

acquire lock

critical section

release lock

remainder section

} while (TRUE);

To be continue ...